

Day (89).

* Pre-Defined character classes :-

→ The pre-defined character classes are defined or designed by Python Lang Developers and available in Python software and whose purpose is to Define/Design search pattern which is used to search, match or find in given data and obtains desired Result.

→ In other words pre-defined character classes are used in defining search patterns.

→ syntax: "\pre-defined character classes"

→ The pre-defined character classes are given below.

- 1.) \d ---> search for digits OR [0-9]
- 2.) \D ---> searches for all except digits OR [^0-9]
- 3.) \w ---> searches word char (Alpha-numerics) only OR [A-Za-z0-9_]
- 4.) \W ---> searches for special symbols (except word char (Alphanumeric)) OR [^A-Za-z0-9_]
- 5.) \s ---> searches for space character.
- 6.) \S ---> searches for all except space character.

gd = "CH4Luap7#Bb6@QpwinDg".
find the letter abc

ex17. `match = re.finditer("\s", "!c t k @ 2 a 4 p 4 # G 9 8 H b Q w")`

"apitals"

ex18)

"small w"

ex19)

"capital w"

ex20)

"small d"

ex21)

"capital b"

ex22)

* Quantifiers in Regular Expressions:-

→ Quantifiers in Regular Expressions are used for searching number of occurrences of the specified value (alphabets or digits or special symbols) used in search pattern to search in the given data and obtain desired result.

→ The Quantifiers in Regular Expressions are:-

- 1) "K" - - - -> Searches for exactly one K.
- 2) K+ - - - -> Searches for one K or more K's
- 3) "K*" - - - -> Searches for zero K or one K or more K's
- 4) "K?" - - - -> Searches for zero K or one K
- 5) "." - - - -> Searching all occurrences of letter of given data.

Special formula:-

- 1) \d+ - - - -> Searching one OR more Digits or [0-9]+
- 2) \d{10} . OR [0-9]{10} - - - -> Searches 10 Digits Exactly.
- 3) \d{m} OR [0-9]{m} - - - -> Searches M-Digit Number.
- 4) \d{m,n} - - - -> Searches for minimum M-Digit Number and maximum n-Digit Number only.

- 1) \w+ - - - -> Searching one OR more word character OR [A-Za-z0-9_]+
- 2) \w{5} . OR [A-Za-z0-9_]{5} - - - -> Searches 5 letter word exactly.
- 3) \w{m} OR [A-Za-z0-9_]{m} - - - -> Searches M-letter word.
- 4) \w{m,n} OR [A-Za-z0-9_]{m,n} - - - -> Searches for minimum M-letter word and maximum n-letter word only.

5) `[A-Za-z]{3}` - - - -> Searches for 3 letters word contains purely Alphabets

6) `\d\d.\d\d` - - - -> Searches for Floating point values contains an integer part with 2 Digits and decimal part with 2 Digits

ex23) `matchab = re.findall("k", "kvvkvvkkkvvkvv")`.

ex24)

`"k+"`

ex25)

`"k*"`

ex26)

`"k?"`

ex27)

`"k."`

ex28) # program for validating mobile Number by using Reg Expr

mobile Number Valid Ex1:

`import re`

`while(True):`

`mno = input("Enter ur mobile number;")`

`if (len(mno) == 10):`

`res = re.search("d{10}", mno)`

`if (res != None):`

`print("\t{} is valid mobile number".format(mno))`

`break`

`else:`

`print("\t{} is invalid bcoz it contains non-digits".format(mno))`

`else:`

`print("\t{} is invalid mobile number due to its invalid length-- try again".format(mno))`

ex29) `But res = re.search("[98]\d{8}", mno)` yam change

program for Extracting the names of students from given text data

ex1).

```
import re
gd = "Rossum is the developer of python, Travis is the
      developer of numpy, Kinney is developer of pandas
      and Ritchie is the developer of C lang"

nameslist = re.findall("[A-Z][a-z]+", gd)
print(nameslist)

for name in nameslist:
    print("\t{}".format(name))

markslist = re.findall("\d{2,3}", gd)
print("marks list")
for marks in markslist:
    print("\t{}".format(marks))
```

ex2).

Yahan the same ~~code~~ code change.

```
markslist = re.findall("\d{2,3}", gd)
print(" — — — ")
print("\tNames\tmarks")
print(" — — — ")
for names, marks in zip(nameslist, markslist):
    print("\t{}\t{}".format(names, marks))
print(" — — — ")
```

→ untitled). Rossum is the developer of python got 56, Travis is the developer of numpy and got 55, Kinney is developer of pandas and got 33 and Ritchie is the developer of C lang, and got 6.6 and Kvr got 11 marks

ex1). Program for extracting student names and marks where in the information present in file (student.data) by using RegEx.

import re

try:

with open ("location", "r") as fp:

filedata = fp.read()

nameslist = re.findall("[A-Z][a-z]+", filedata)

markslist = re.findall("\d{2}", filedata)

print(" - - - ")

print("\tNames\tMarks")

print(" - - - - ")

for names, marks in zip(nameslist, markslist):

print("\t{}\t{}".format(names, marks))

print(" - - - ")

except FileNotFoundError:

print("file does not exist")

#untitled). Rossum is the developer of python and whose mail is rossum.python@psf.com, Travis is the developer of numpy mail is travis12@nompv.org, Kinney is developer of pandas and mail id is kinney_wes@pandas.org and Ritchie is the developer of clang mail id is ritche.c@bellabs.net and kvr mail id is kvr.python@gmail.com

ex1). program for extracting the mail from file (email.data).

try: with open ("location", "r") as fp:

maildata = fp.read()

Extract mails

mailslst = re.findall("\S+@\S+", maildata)

nameslist = re.findall("[A-Z][a-z]+", maildata)

sp=re.compile("\S+@\S+") print(" - - - ")

print("\tNames\tmail-id")

print(" - - - ")

```

for name, mails in zip(namelist, mailslist):
    print("\t{0}\t{1}:".format(name, mails))
print(" - - - ")

except FileNotFoundError:
    print("file does not exist")

```

* Working JSON files (JavaScript object notation): -

- JSON stands for java script object notation.
- JSON file format is a language independent concept and it can be used in all the languages bcoz JSON file format is one of the light weight file format in data exchanging b/w client and server side application in the internet world (web application development).
- since JSON file format exchanging data between client and server side application in the form of (key, value) and it is called dictionary and in python it is related dict data type.
- To take any information in the form json file, it must save on some file name with extension .json (filename.json) where it contains (key, value).
- To implement JSON file format in python programming, we must use a pre-defined module called "json".
- In python programming, JSON file format is shown below:-

```

Varname = { "key1": "val", "key2": "val2"
            ... "key-n": "val-n" }

```

* Function in json module:

1) Parse JSON (Convert from JSON str data to python dict).

=> Json.loads() function can parse a json string and converted into python dictionary.

syntax) dictobj = json.loads(json_string).

Examples).

Python program to JSON to Python.

```
import json
```

JSON string

employee = { "id": "09", "name": "Rossum",
"department": "IT" }

Convert JSON string to python dict

```
employee_dict = json.loads(employee)
```

```
print(employee_dict).
```

— Python -- read JSON file data.

⇒ `json.load()` function can read the data from JSON file which contains a JSON data and placed in dict data.

Syntax) dictObj = json.load (file_pointer)

Example).

program for JSON file Data into dict object

Json Filedict.py - - - reading the data from json file to Dictobj

import json

try: with open("emp.json", "r") as fp:

$$d(\text{Ctoobj}) = \gamma \text{sm} \cdot \text{load}(\text{fp})$$

```
print(dictobj, type(dictobj))
```

```
print(" - - - ")
```



```

for k,v in dictobj.items():
    print("\t{}-->{}".format(k,v))
print(" - - - ")
except FileNotFound error:
    print("JSON file does not exist")

— Python ——— write Dict Data to JSON file
syntax). json.dump(dict object, file pointer)

example).
# Program for Dict data into JSON file
# Dict to JSON File.py ——— writing Dict data to JSON file

import json
dictobj = { "eno": 100, "ename": "Pravis", "sal": 56, "Dsg":
            "Author" }

with open("emp.json", "w") as fp:
    json.dump(dictobj, fp) # Here dump() is
                           saving dictobj data into
                           the json file
    print("Dict Data saved in JSON file format -- verify")

```

```

In [1]: #Searches for all except digits
#RegExpr17
import re
matd=re.finditer(r"\D","!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")

```



```
-----
Start Index:0 End Index:1 Value:!
Start Index:1 End Index:2 Value:c
Start Index:2 End Index:3 Value:H
Start Index:4 End Index:5 Value:L
Start Index:5 End Index:6 Value:u
Start Index:6 End Index:7 Value:a
Start Index:7 End Index:8 Value:P
Start Index:9 End Index:10 Value:#
Start Index:10 End Index:11 Value:B
Start Index:11 End Index:12 Value:b
Start Index:13 End Index:14 Value:@
Start Index:14 End Index:15 Value:Q
Start Index:15 End Index:16 Value:p
Start Index:16 End Index:17 Value:W
Start Index:17 End Index:18 Value:m
Start Index:18 End Index:19 Value:D
Start Index:20 End Index:21 Value:%
-----
```

```
In [2]: #Searches for all word character
#RegExpr18
import re
matd=re.finditer(r"\w","!cH4LuaP7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:1 End Index:2 Value:c
Start Index:2 End Index:3 Value:H
Start Index:3 End Index:4 Value:4
Start Index:4 End Index:5 Value:L
Start Index:5 End Index:6 Value:u
Start Index:6 End Index:7 Value:a
Start Index:7 End Index:8 Value:P
Start Index:8 End Index:9 Value:7
Start Index:10 End Index:11 Value:B
Start Index:11 End Index:12 Value:b
Start Index:12 End Index:13 Value:6
Start Index:14 End Index:15 Value:Q
Start Index:15 End Index:16 Value:p
Start Index:16 End Index:17 Value:W
Start Index:17 End Index:18 Value:m
Start Index:18 End Index:19 Value:D
Start Index:19 End Index:20 Value:9
-----
```

```
In [3]: #Searches for all special symbols
#RegExpr19
import re
matd=re.finditer(r"\W","!cH4LuaP 7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:!
Start Index:8 End Index:9 Value:
Start Index:10 End Index:11 Value:#
Start Index:14 End Index:15 Value:@
Start Index:21 End Index:22 Value:%
-----
```

```
In [4]: #Searches for all Space Characters
#RegExpr20
import re
matd=re.finditer(r"\s","!cH 4LuaP 7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:3 End Index:4 Value:
Start Index:9 End Index:10 Value:
-----
```

```
In [5]: #Searches for all except Space Characters
#RegExpr21
import re
matd=re.finditer(r"\S","!cH 4LuaP 7#Bb6@QpWmD9%")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.en
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:!
Start Index:1 End Index:2 Value:c
Start Index:2 End Index:3 Value:H
Start Index:4 End Index:5 Value:4
Start Index:5 End Index:6 Value:L
Start Index:6 End Index:7 Value:u
Start Index:7 End Index:8 Value:a
Start Index:8 End Index:9 Value:P
Start Index:10 End Index:11 Value:7
Start Index:11 End Index:12 Value:#
Start Index:12 End Index:13 Value:B
Start Index:13 End Index:14 Value:b
Start Index:14 End Index:15 Value:6
Start Index:15 End Index:16 Value:@
Start Index:16 End Index:17 Value:Q
Start Index:17 End Index:18 Value:p
Start Index:18 End Index:19 Value:W
Start Index:19 End Index:20 Value:m
Start Index:20 End Index:21 Value:D
Start Index:21 End Index:22 Value:9
Start Index:22 End Index:23 Value:%
-----
```

```
In [6]: #Searches for exactly one K
#RegExpr22
import re
matd=re.finditer("K","KVKKVKKKVKV")
print("-----")
for mat in matd:
```

```
print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:K
Start Index:2 End Index:3 Value:K
Start Index:3 End Index:4 Value:K
Start Index:5 End Index:6 Value:K
Start Index:6 End Index:7 Value:K
Start Index:7 End Index:8 Value:K
Start Index:9 End Index:10 Value:K
-----
```

```
In [7]: #Searches for one K or More K's
#RegExpr23
import re
matd=re.finditer("K+","KVKKKVKKV")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:K
Start Index:2 End Index:4 Value:KK
Start Index:5 End Index:8 Value:KKK
Start Index:9 End Index:10 Value:K
-----
```

```
In [8]: #Searches for Zero K or one K or More K's
#RegExpr24
import re
matd=re.finditer("K*","KVKKKVKKV")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:K
Start Index:1 End Index:1 Value:
Start Index:2 End Index:4 Value:KK
Start Index:4 End Index:4 Value:
Start Index:5 End Index:8 Value:KKK
Start Index:8 End Index:8 Value:
Start Index:9 End Index:10 Value:K
Start Index:10 End Index:10 Value:
Start Index:11 End Index:11 Value:
-----
```

```
In [9]: #Searches for Zero K or one K
#RegExpr25
import re
matd=re.finditer("K?","KVKKKVKKV")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```



```
-----
Start Index:0 End Index:1 Value:K
Start Index:1 End Index:1 Value:
Start Index:2 End Index:3 Value:K
Start Index:3 End Index:4 Value:K
Start Index:4 End Index:4 Value:
Start Index:5 End Index:6 Value:K
Start Index:6 End Index:7 Value:K
Start Index:7 End Index:8 Value:K
Start Index:8 End Index:8 Value:
Start Index:9 End Index:10 Value:K
Start Index:10 End Index:10 Value:
Start Index:11 End Index:11 Value:
-----
```

```
In [10]: #Searches for all values
#RegExpr26
import re
matd=re.finditer(".", "KVKKVKKKKVKV")
print("-----")
for mat in matd:
    print("Start Index:{} End Index:{} Value:{}".format(mat.start(),mat.end(),mat.group()))
print("-----")
```

```
-----
Start Index:0 End Index:1 Value:K
Start Index:1 End Index:2 Value:V
Start Index:2 End Index:3 Value:K
Start Index:3 End Index:4 Value:K
Start Index:4 End Index:5 Value:V
Start Index:5 End Index:6 Value:K
Start Index:6 End Index:7 Value:K
Start Index:7 End Index:8 Value:K
Start Index:8 End Index:9 Value:V
Start Index:9 End Index:10 Value:K
Start Index:10 End Index:11 Value:V
-----
```

```
In [11]: #Program for Validating Mobile Number
#MobileNumberValidEx1
import re
while(True):
    mno=input("Enter Ur Mobile Number:")
    if (len(mno)==10):
        res=re.search(r"\d{10}",mno)
        if(res!=None):
            print("{} is Valid Mobile Number".format(mno))
            break
        else:
            print("\tMobile Number Should Not Contain Non-Ints--try again")
    else:
        print("\t{} is Invalid mobile Number due to wrong length-try again")
```

is Invalid mobile Number due to wrong length-try again
7644824622 is Valid Mobile Number

```
In [12]: #Program for extracting Names and Marks of students from given text
#NamesMarksExtractEx1
import re
gd="Rossum got 55 marks , Travis got 44 marks , Kinney got 66 marks and Gosling got 77 marks"
sp=r"\d{2}"
```

```

markslist=re.findall(sp,gd)
nameslist=re.findall("[A-Z][a-z]+",gd)
print("-----")
print("Names\tMarks")
print("-----")
for names,marks in zip(nameslist,markslist):
    print("{}\t{}".format(names,marks))
print("-----")

```

```

-----
Names    Marks
-----

```

```

Rossum   55
Travis   44
Kinney   66
Gosling  22
Kvr      11
-----

```

In [14]: *#Program for extracting Marks of students from given text*
#MarksExtractEx1

```

import re
gd="Rossum got 55 marks , Travis got 44 marks , Kinney got 66 marks and Gosling
sp=r"\d{2}"
markslist=re.findall(sp,gd)
print("-----")
print("Marks")
print("-----")
for marks in markslist:
    print(marks)
print("-----")

```

```

-----
Marks
-----

```

```

55
44
66
22
11
-----

```

In [15]: *#DictToJsonFile.py*

```

import json
d={'sno': '100', 'name': 'ROSSUM', 'marks': '23.45'}
print(d,type(d))
with open("stud.json","a") as fp:
    json.dump(d,fp) # Transfer Dict Data to JSON File
    print("Diict Data Saved in Json File--verify")

```

```

{'sno': '100', 'name': 'ROSSUM', 'marks': '23.45'} <class 'dict'>
Diict Data Saved in Json File--verify

```

In [16]: *#JsonFileToDict.py*

```

import json
with open("stud.json","r") as fp:
    d=json.load(fp)
    print(d,type(d))
    print("-----")
    for k in d:
        print("\t{}-->{}".format(k,d[k]))

```

```

print("-----")
for k in d:
    print("\t{}-->{}".format(k, d.get(k)))
print("-----")
for k,v in d.items():
    print("\t{}-->{}".format(k, v))
print("-----")
#Reverting the Dict Data
rd={v:k for k,v in d.items()}
for k,v in rd.items():
    print("\t{}-->{}".format(k, v))

```

```
{'sno': '100', 'name': 'ROSSUM', 'marks': '23.45'} <class 'dict'>
```

```

-----
sno-->100
name-->ROSSUM
marks-->23.45
-----
sno-->100
name-->ROSSUM
marks-->23.45
-----
sno-->100
name-->ROSSUM
marks-->23.45
-----
100-->sno
ROSSUM-->name
23.45-->marks

```

```

In [17]: #JsonToDict.py
import json
jsonstrdata='{"sno":"100","name":"ROSSUM","marks":"23.45"}'
print(jsonstrdata,type(jsonstrdata))
print("-----")
#Convert Json Str Data into Dict Object
dobj=json.loads(jsonstrdata)
for key,val in dobj.items():
    print("\t{}-->{}".format(key,val))
print("-----")
#Again Convert Dict Object into str format by using str()
sdict=str(dobj)
print(sdict,type(sdict))

```

```
{"sno":"100","name":"ROSSUM","marks":"23.45"} <class 'str'>
```

```

-----
sno-->100
name-->ROSSUM
marks-->23.45
-----

```

```
{'sno': '100', 'name': 'ROSSUM', 'marks': '23.45'} <class 'str'>
```

```

In [20]: #Program for Listing the Files in a Folder
#ListFilesEx2
import os
try:
    FolderName=input("Enter the Folder Name to List the Files:")
    FileList=os.listdir(FolderName)
    print("-----")
    for file in FileList:
        print("\t{}".format(file))

```

```
print("-----")
print("Number of Files=", len(FileList))
print("-----")
except FileNotFoundError:
    print("Folder Does not Exist")
```

Folder Does not Exist

In []: